UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

SE 423: Introduction to Mechatronics

Lecture 18: Path planning

Marius Juston

Monday, March 30th, 2026

Review – 5-minute partner discussion

Talk to the person next to you and answer these questions.

- Last lecture we said we used “BFS”, what does it stand for? What did we use it for?
- Open on the lecture’s website the path planning visualization that was made for this lecture (https://marius-juston.github.io/SE-423---Class-Material/path_planning.html), play around and draw some things and press the run button. You don’t need to understand what A* Search, BFS, Dijkstra's is yet, but I would recommend having this up during lecture so that you can see the different algorithms.

Reminders

Office Hours:

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

Lab: Starting Lab 6 this week. This a 3-week lab!

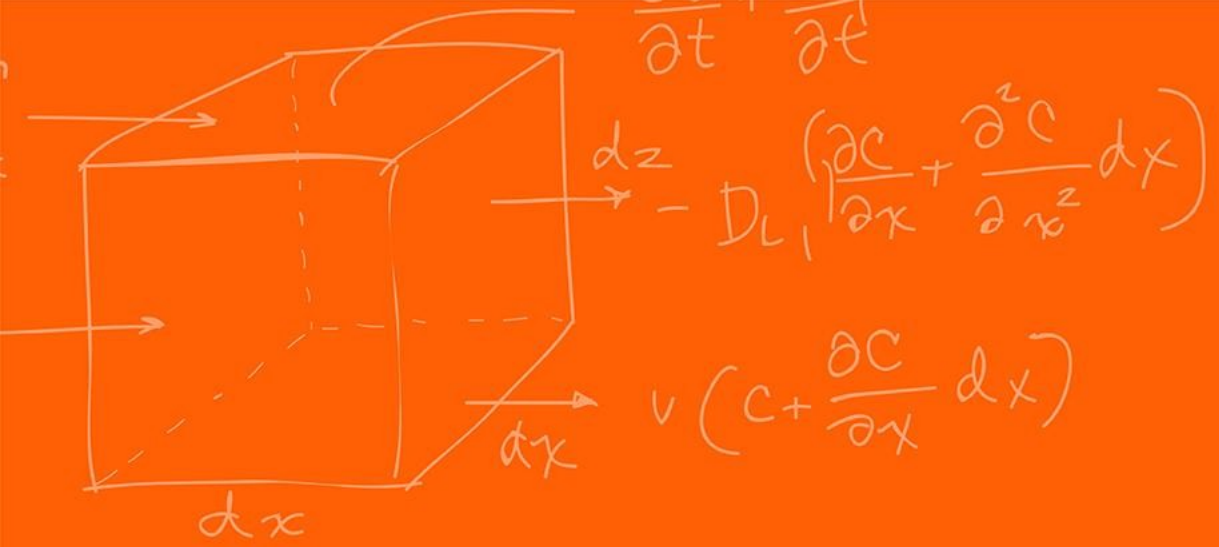
- Don’t forget to have the Lab LabVIEW application implemented before lab.

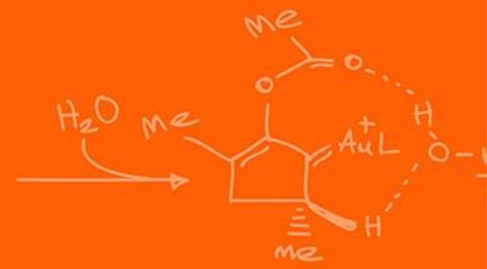
Homeworks:

- Check-offs for homework 4 Ex 2 and 3 are due **March 31, 5PM (The end of office hours)**
- Check-offs for homework 4 Ex 4 are due **April 7, 5PM (The end of office hours)**
- Remainder of homework 4 due on Gradescope on **April 8, 9AM**

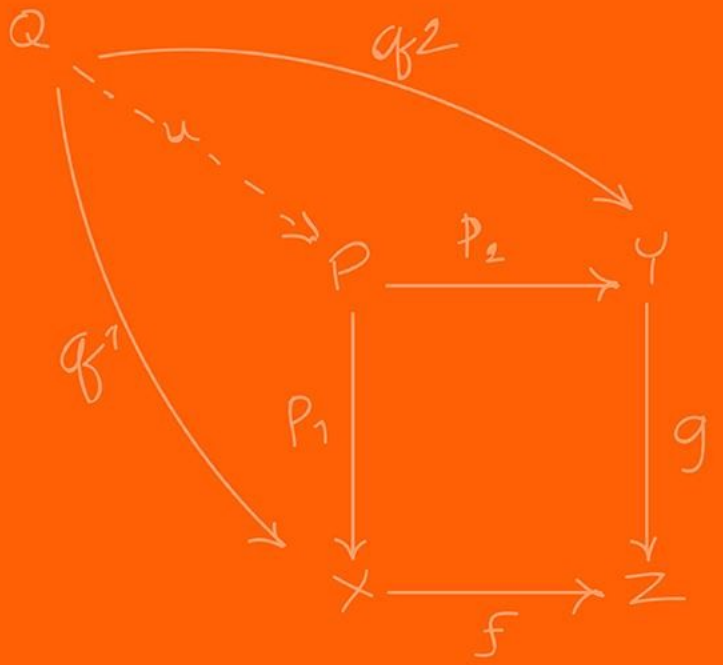
Questions?

Homework, Lab, LabVIEW?





Path planning

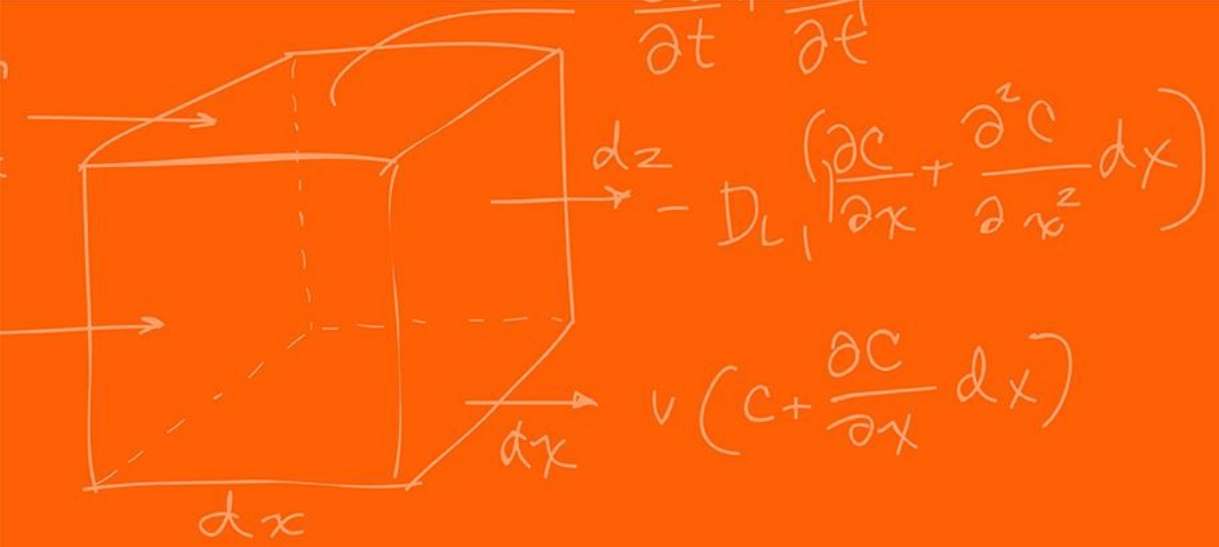


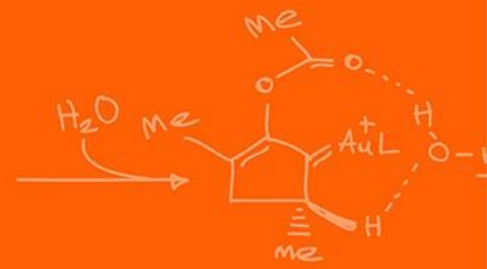
Preface

I would HIGHLY recommend you looking at:

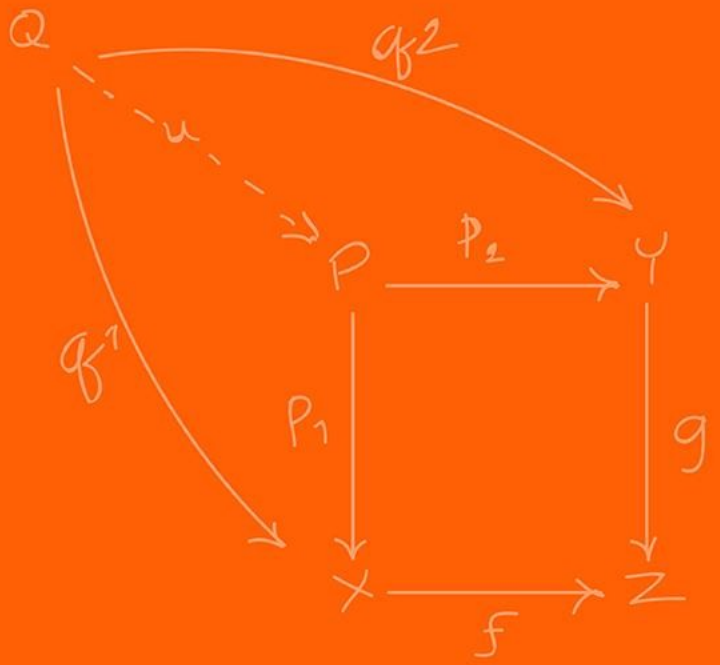
- https://theory.stanford.edu/~amitp/GameProgramming/MapRepresentations.html
- https://www.redblobgames.com/pathfinding/a-star/introduction.html

This is a series of interactable tutorial for path planning, it is very useful and goes through all the material in this lecture!





Graphs

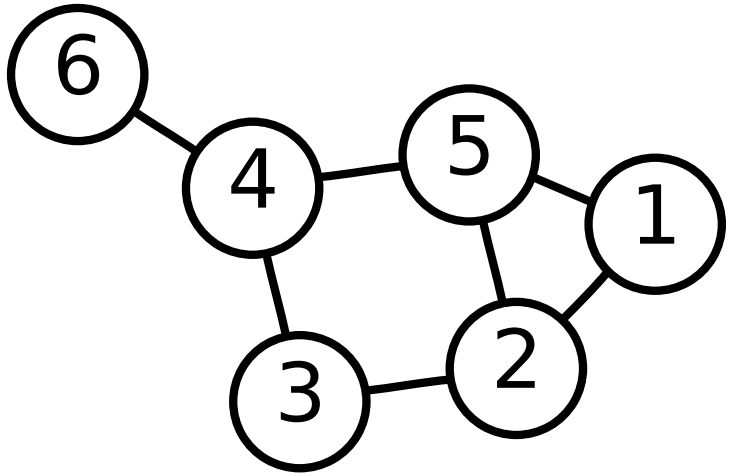


Graphs

Discrete path planning works on a graph data structure, it can be directed or undirected.

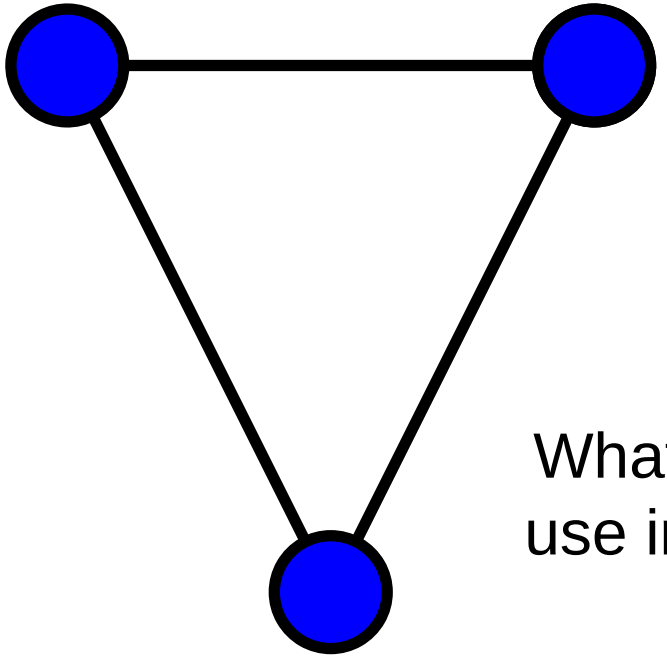
A graph is composed of 2 simple components:

- Edges - E: Paths or transitions between nodes
- Vertices (Nodes) - V: Locations, states, or positions in space
- Weights (Costs) - $w(e)$: Distance, time, energy or terrain difficult to traverse specific edge

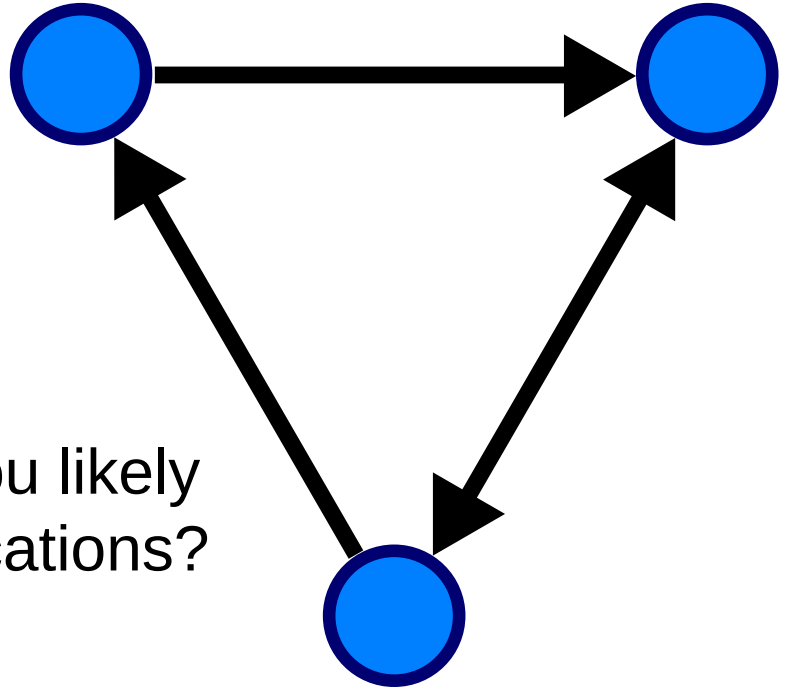


Graphs

- An undirected (or simple) graph has edges that are bi-directions, you can to and from vertices where nodes are connected

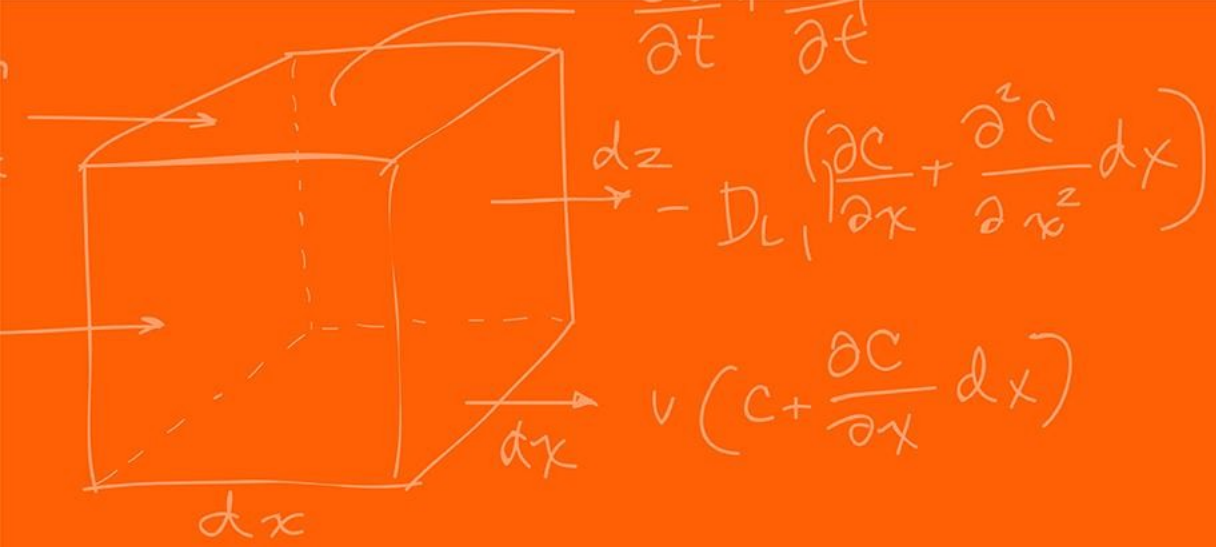


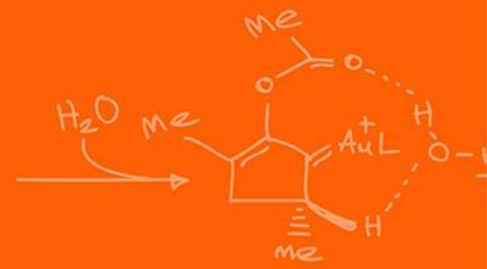
- A directed graph has edges that have a direction, you can only go from “a” to “b” and not “b” to “a”

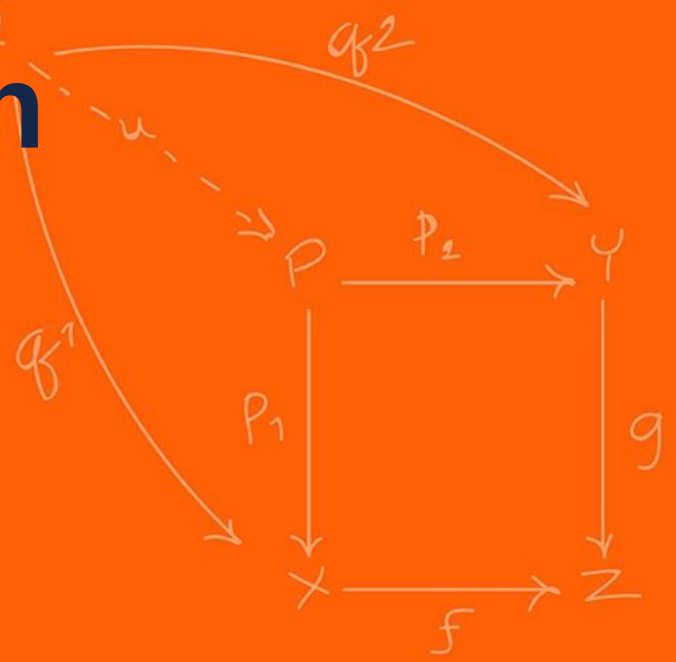


What type would you likely use in robotic applications?

Map Representation







Graphs

Because we live in a Euclidian space, we can often represent the locations that the robot can move to in discrete 2D as a **2D map**.

The map representation can make a huge difference in the performance and path quality.

Pathfinding algorithms tend to be worse than linear (time complexity): if you double the distance needed to travel, it takes more than twice as long to find the path.

You can think of pathfinding as searching some area like a circle—when the circle's diameter doubles, it has four times the area.

Graphs

In general, the fewer nodes in your map representation, the faster the path planning will be.

Also, the more closely your nodes match the positions that units will move to, the better your path quality will be.

The best map representation is defined based on the data that you have, and map visibility.

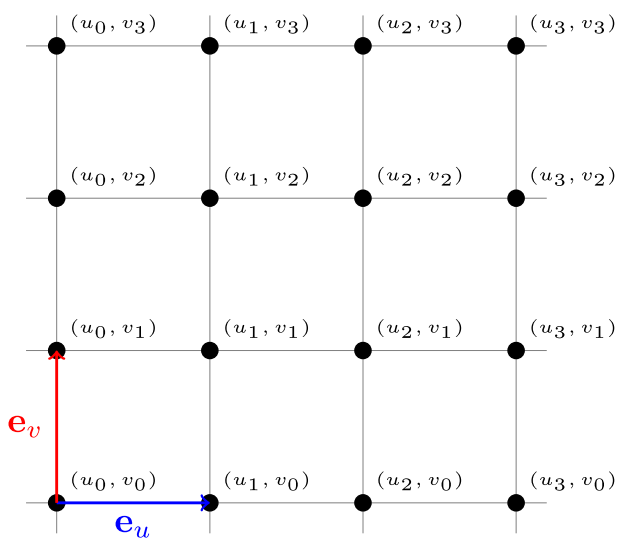
Graphs

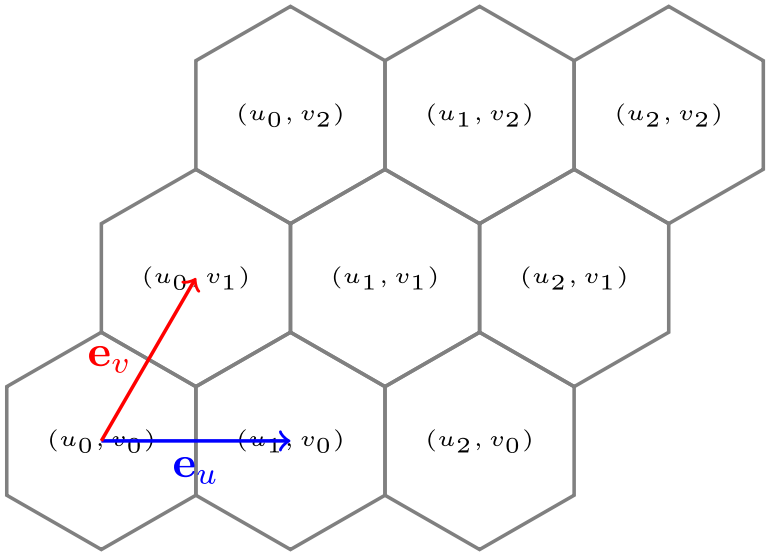
A grid map uses a uniform subdivision of the world into small regular shapes sometimes called “tiles”. Common grids in use are **square**, **triangular**, and **hexagonal**.

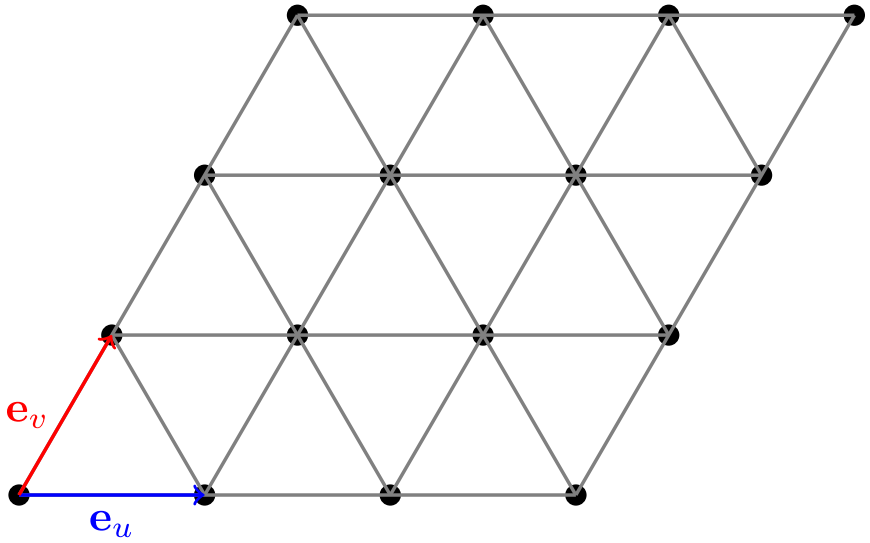
Grids are simple and easy to understand, and many games and robotic applications use them for world representation.

With grids you usually move between tiles, where the movement costs are uniform.

Graphs


(u_0, v_3)
(u_1, v_3)
(u_2, v_3)
(u_3, v_3)
(u_0, v_2)
(u_1, v_2)
(u_2, v_2)
(u_3, v_2)
(u_0, v_1)
(u_1, v_1)
(u_2, v_1)
(u_3, v_1)
(u_0, v_0)
(u_1, v_0)
(u_2, v_0)
(u_3, v_0)
$\mathbf{e}_v$
$\mathbf{e}_u$

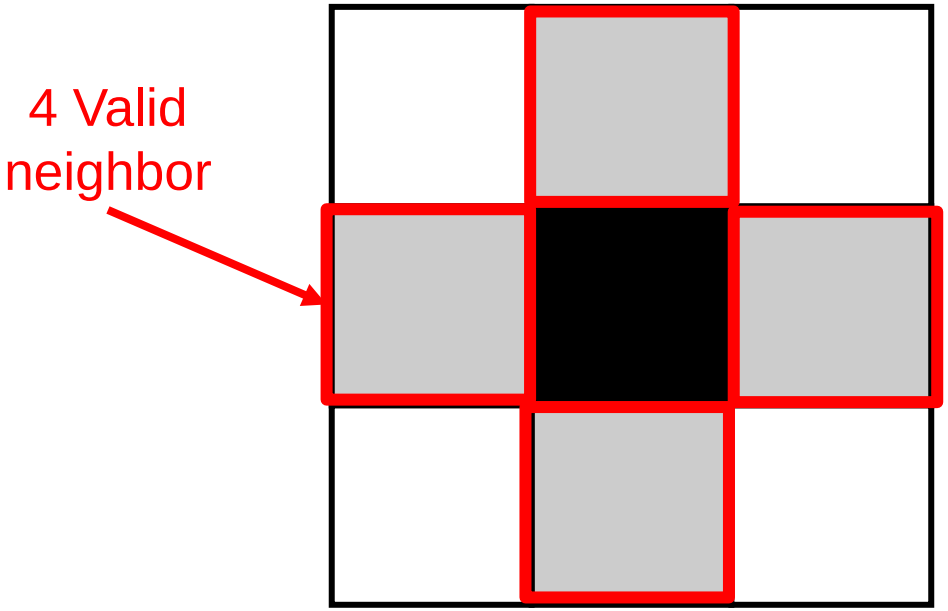

(u_0, v_2)
(u_1, v_2)
(u_2, v_2)
(u_0, v_1)
(u_1, v_1)
(u_2, v_1)
(u_0, v_0)
(u_1, v_0)
(u_2, v_0)
$\mathbf{e}_v$
$\mathbf{e}_u$

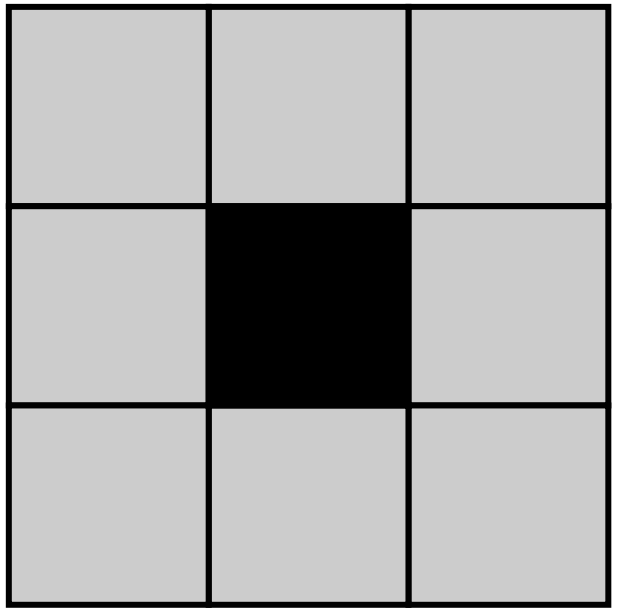

$\mathbf{e}_v$
$\mathbf{e}_u$

Graphs

In grid spaces, we have something called connectivity, the most used is 4-connected neighbors and 8-connected components.

Neighbor connectivity informs us which neighbors we could potentially reach at the next time step.
Which is 4-connected neighbors?





Graphs

What is are some potential problems with grid maps?

Based on the resolution of the grid and the total area of the grid the following happens:

- Requires quadratic amount of memory per resolution increase
- The time it takes to solve the path is based on the resolution and the area
- A lot of the grid is “redundant” information, adjacent neighbors do not advance the robot in a substantial way

However, it is extremely simple to implement! It is just a 2D (or 1D) array!

Graphs

The most common alternative to grids is to use a polygonal representation.

If the movement cost across large areas is uniform, and if your units can move in straight lines instead of following a grid, you may want to use a non-grid representation.

You can use a non-grid graph for pathfinding even if your game uses a grid for other things.

Graphs

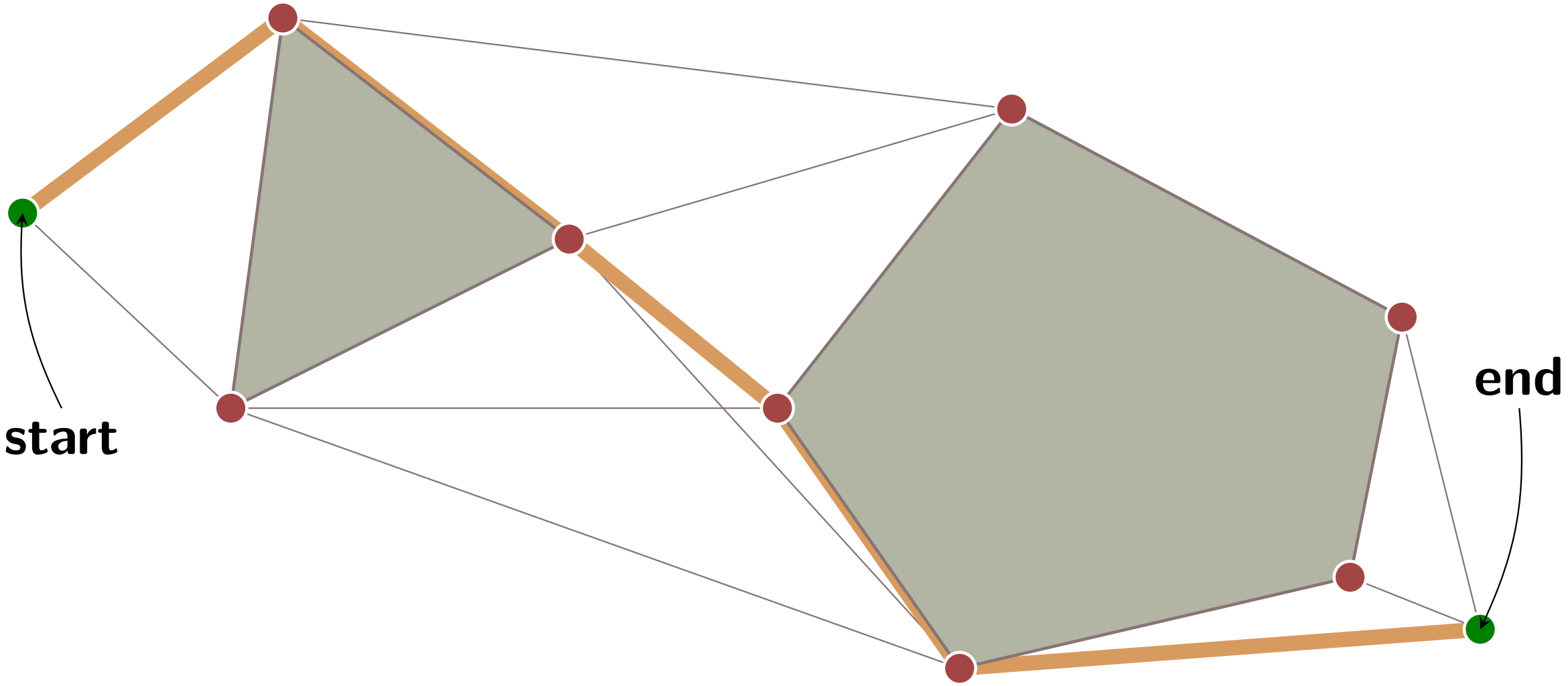
The shortest path will be between corners of the obstacles.

So, we choose those corners as the key “**navigation points**” points; these can be computed once per map change.

If your obstacles are aligned on a grid, the navigation points will be aligned with the vertices of the grid.

In addition, the start and end points for pathfinding need to be in the graph; these are added once per navigation task.

Graphs



Graphs

In addition to the navigation points, we need to know which points are **connected**.

The simple algorithm is to build a **visibility graph**: pairs of points that can be seen from each other without colliding with the obstacles.

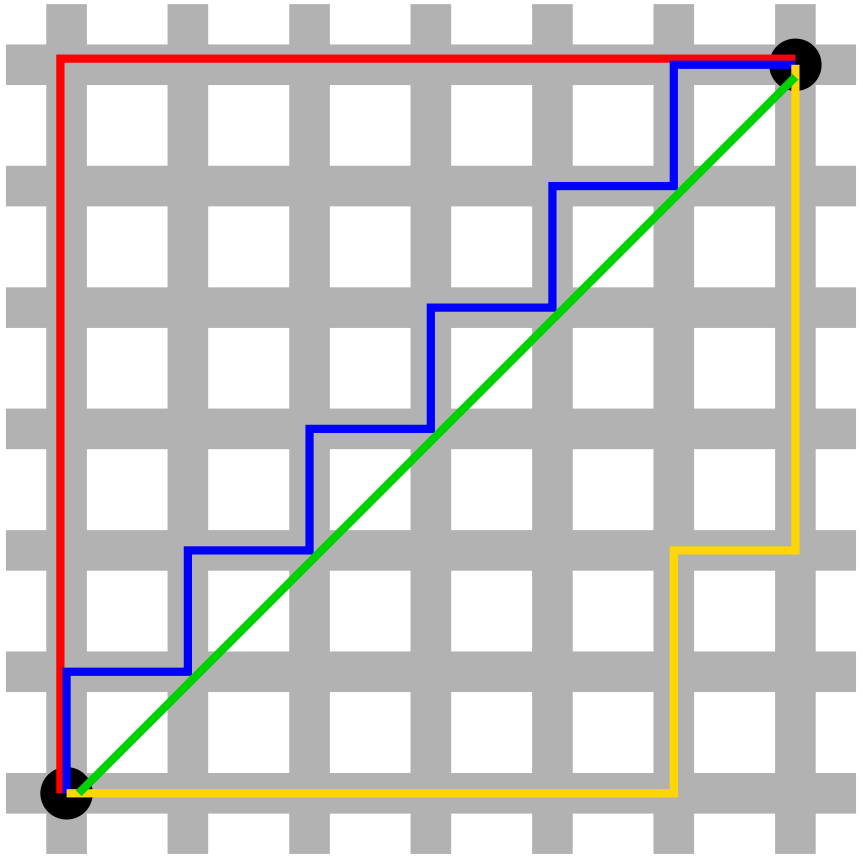
However, this representation increase time complexity but potentially greatly reduces the search space.

The third piece of information we needs is travel times between the points. That will be **Manhattan distance** or **diagonal grid** distance if your units move on a grid, or **straight-line** distance if they can move directly between the navigation points.

Because we work on rectangular grids, we can define different **distance metrics**.

We are traveling on this grid, order which is the shortest path distance-wise?

Which gives the closest travel distance when traveling on the grid?



The green is the shortest ($6\sqrt{2}$), all the others are the same distance (12)

The red, blue and yellow, since we cannot move diagonally

Graphs

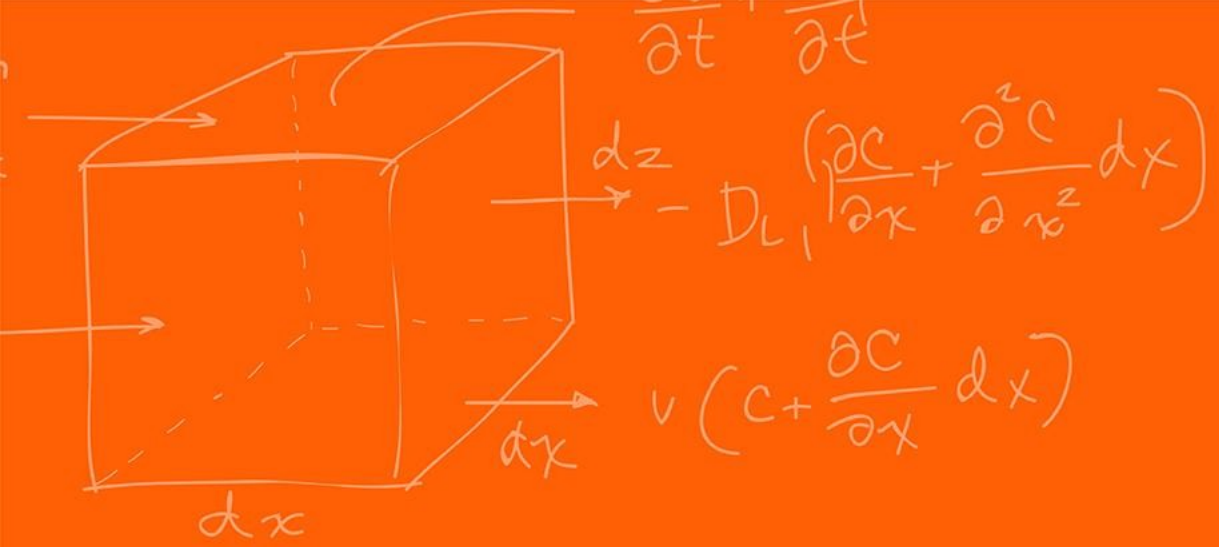
The most common distance metrics in path planning are **Euclidian**:

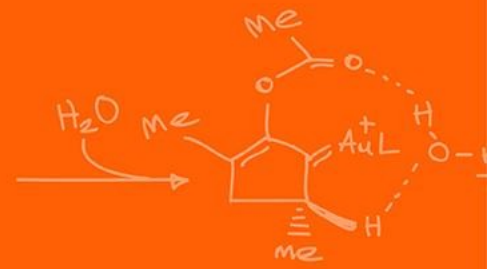
$$d(p,q) = \|\mathrm{p} - \mathrm{q}\|_2 = \sqrt{\sum_{i=1}^{n}(p_i - q_i)^2}$$

Or **Manhattan** (also known as Taxicab):

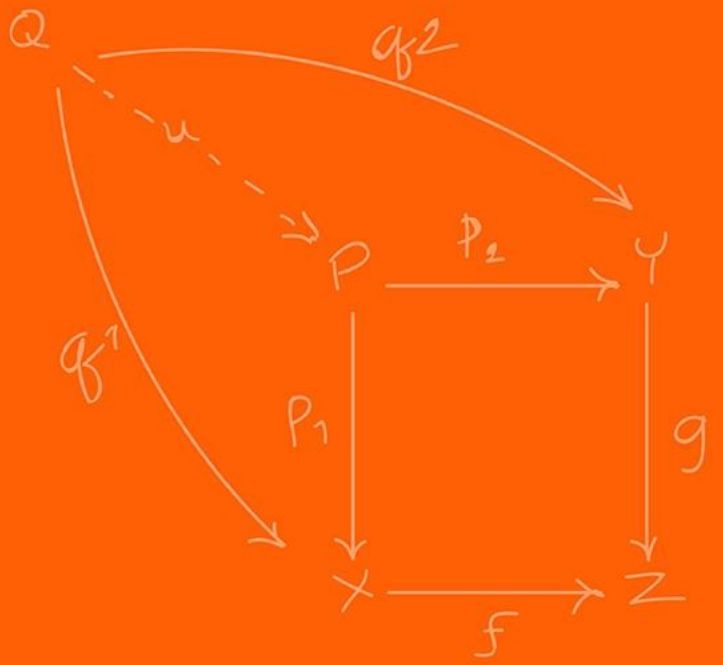
$$d(p,q) = \|\mathrm{p} - \mathrm{q}\|_1 = \sum_{i=1}^{n}|p_i - q_i|$$

To have the most representative path planner you need to ensure the correct distance metric for your grid / task.





Path Planning



Path Planning

Once we have a graph representation of the environment, we now want to find the **“shortest” path**!

The shortest path is not necessarily what we always want. In what real-world situation would not want to take the shortest path?

- Go up climb up a mountain to go to the other side of it instead of just going around it.
- Driving in mud or water vs dry road for safety.

What we actually want is the **lowest-cost path!**

Path Planning - BFS

The key idea for all of these algorithms is that we keep track of an expanding ring called the **frontier**.

On a grid, this process is sometimes called “flood fill”, but the same technique also works for non-grids.

The simplest path planning algorithm that uses this is BFS (Breadth First Search),

Path Planning - BFS

It is incredibly simple to implement **BFS**, only 10 lines of code!

```
frontier = Queue()
frontier.put(start )
reached = set()
reached.add(start)

while not frontier.empty():
    current = frontier.get()
    for next in graph.neighbors(current):
        if next not in reached:
            frontier.put(next)
            reached.add(next)
```

Path Planning - BFS

But how do we find the shortest path?

The loop doesn’t actually construct the paths; it only tells us how to visit everything on the map.

That’s because Breadth First Search can be used for a lot more than just finding paths; for example, in the previous lecture it was used for connected component finding for “blob” search.

Here though we want to use it for finding paths, so let’s modify the loop to keep track of where we came from for every location that’s been reached, and rename the **reached** set to a **came_from** table (the keys of the table are the reached set)

Path Planning - BFS

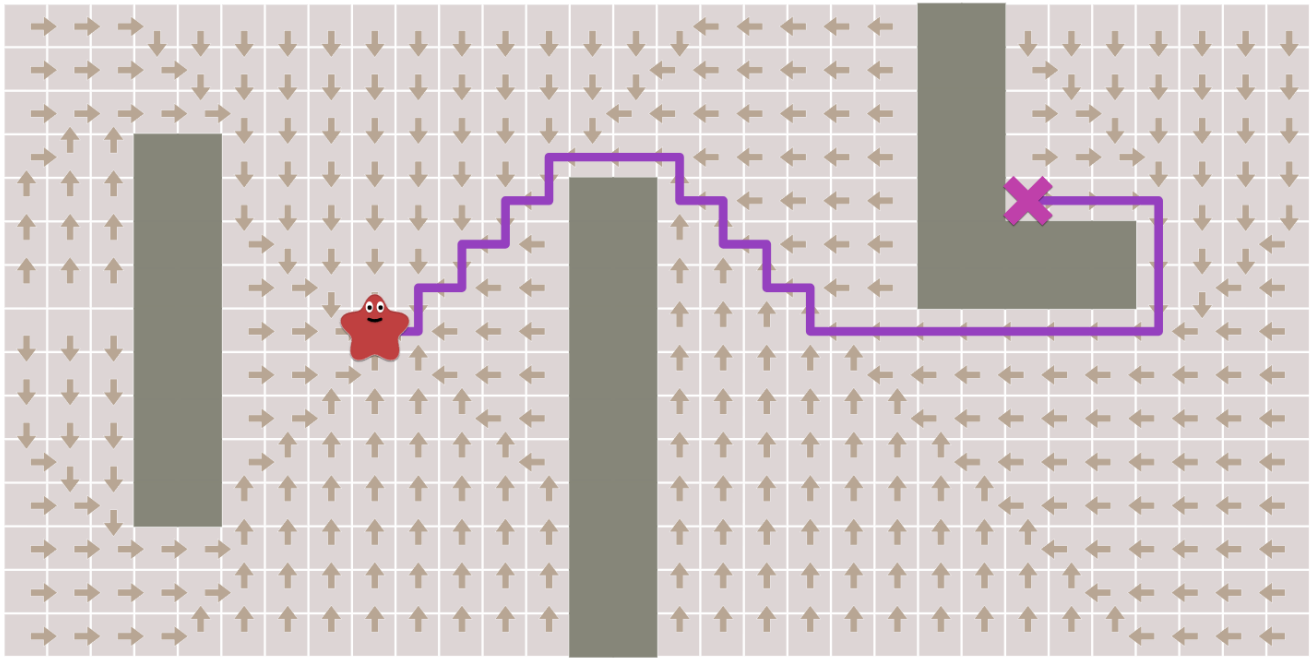
```
frontier = Queue()
frontier.put(start )
came_from = dict() # path A->B is stored as came_from[B] == A
came_from[start] = None

while not frontier.empty():
   current = frontier.get()
   for next in graph.neighbors(current):
      if next not in came_from:
         frontier.put(next)
         came_from[next] = current
```

Path Planning - BFS

Now **came_from** for each location points to the place where we came from.

They form a “flow field” and act like “**breadcrumbs**”.

They’re enough to reconstruct the entire path!



Path Planning - BFS

To reconstruct paths: *follow the arrows backwards **from** the goal **to** the start*. A path is a *sequence of edges*, but often it's easier to store the nodes:

```
current = goal
path = []
while current != start:
    path.append(current)
    current = came_from[current]
path.append(start) # optional
path.reverse() # optional
```

That's the simplest pathfinding algorithm. It works not only on grids as shown here but on any sort of graph structure

Path Planning – Early Exit

We've found paths from one location to **all other locations**. Often, we don't need all the paths; we only need a path from one location to one other location.

We can **stop** expanding the frontier as soon as we've **found our goal!**

Early exit does not necessarily need to be based on when you reach your goal what are other early termination criteria's?

- Distance limit
- Number of goals reached
- Area limit (number of tiles reached)

Path Planning – Movement Cost

- So far, we've made steps have the same "**cost**".

In some pathfinding scenarios there are different costs for different types of movement.

One example is diagonal movement on a grid that costs more than axial movement (1 vs $\sqrt{2}$).

We'd like the pathfinder to take these costs into account.

Path Planning – Dijkstra's Algorithm

For this we want Dijkstra's Algorithm (or Uniform Cost Search).

How does it differ from Breadth First Search?

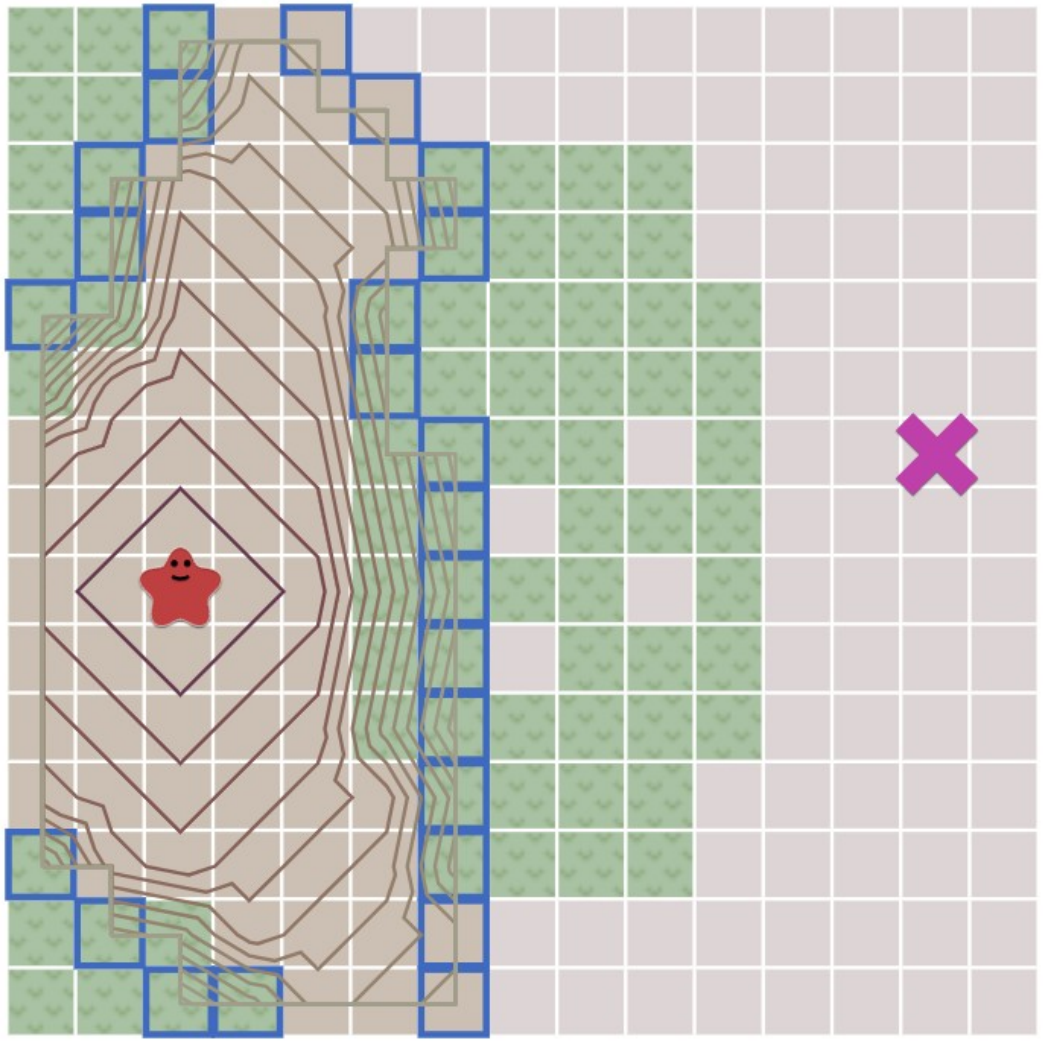
We need to track movement costs, so let's add a new variable, **cost_so_far**. This stores the total movement cost from the start location, also called a "**distance field**".

We want to take the movement costs into account when deciding how to evaluate locations; let's turn our queue into a **priority queue**.

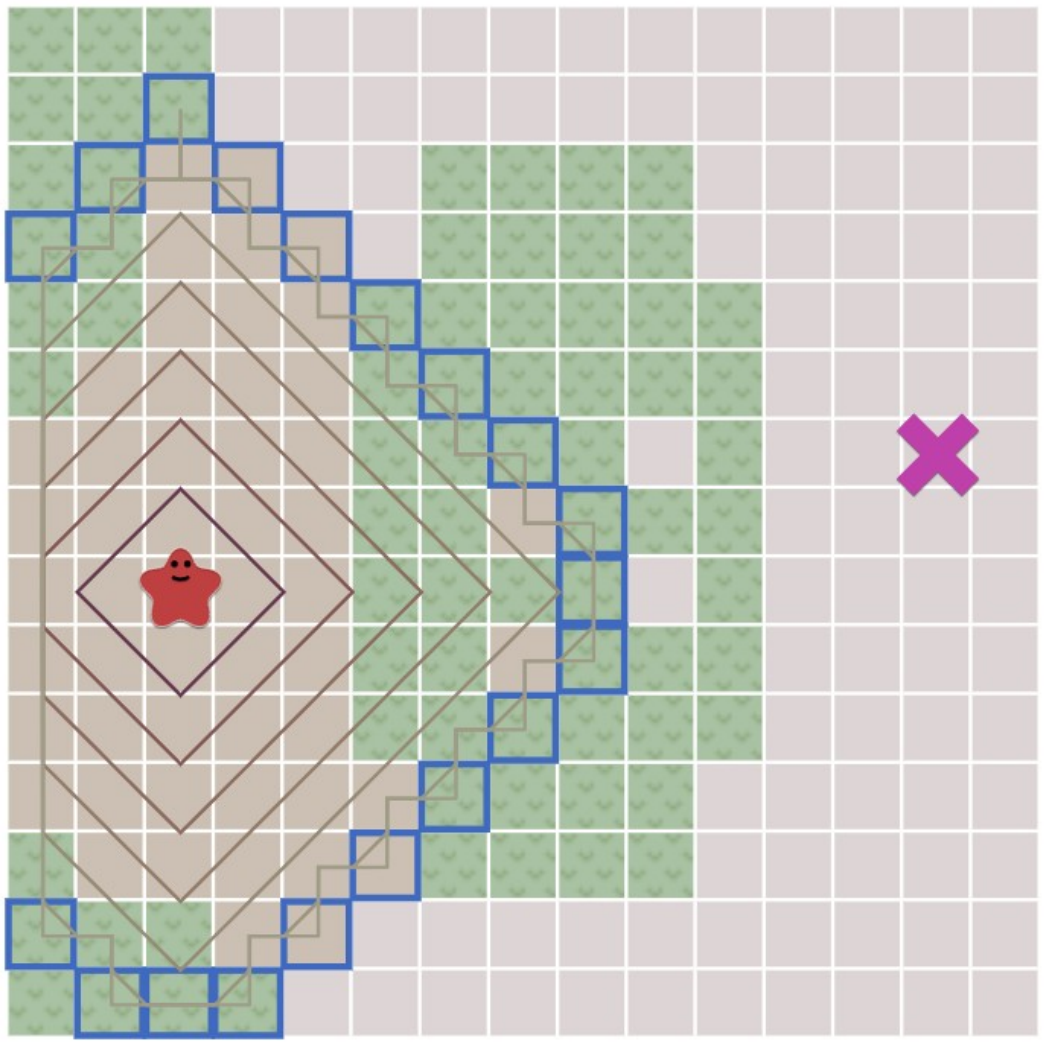
A **priority queue** (min priority queue), is like a **queue** but instead of first-in-first-out, you get lowest-cost-out! This changes how the **frontier** expands!

Path Planning – Movement Cost

Dijkstra's Algorithm



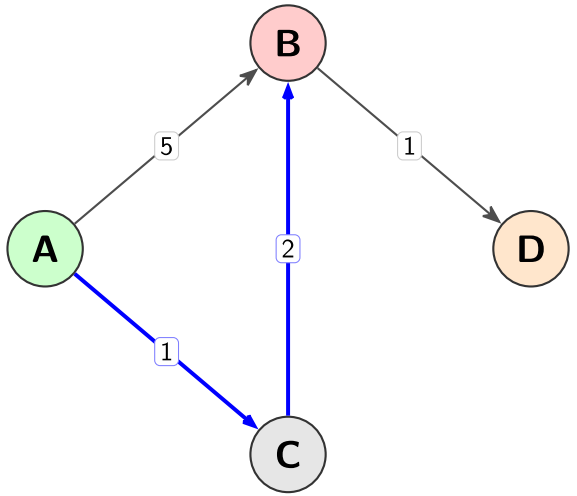
Breadth First Search



Path Planning – Movement Cost

Less obviously, we may end up **visiting a location multiple times**, with different costs, so we need to alter the logic a little bit.

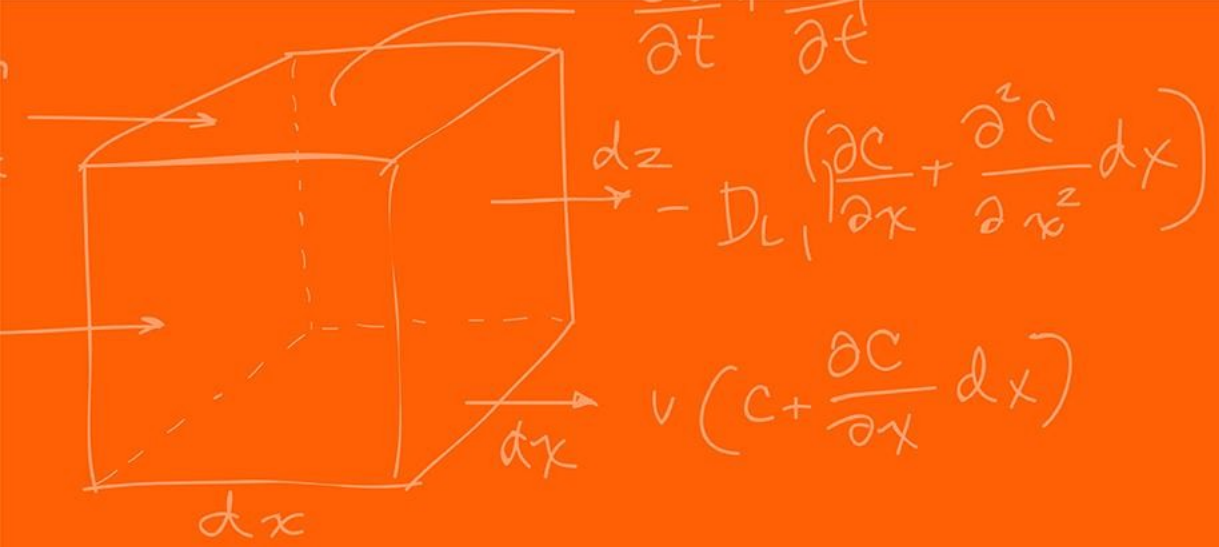
This is because you can have 2 paths to reach one node with the same unit distance but different costs.

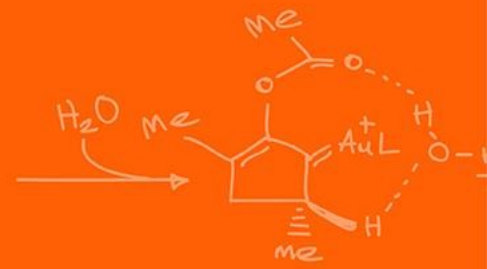


Instead of adding a location to the frontier if the location has never been reached, we'll add it if the new path to the location is **better than the best previous path**.

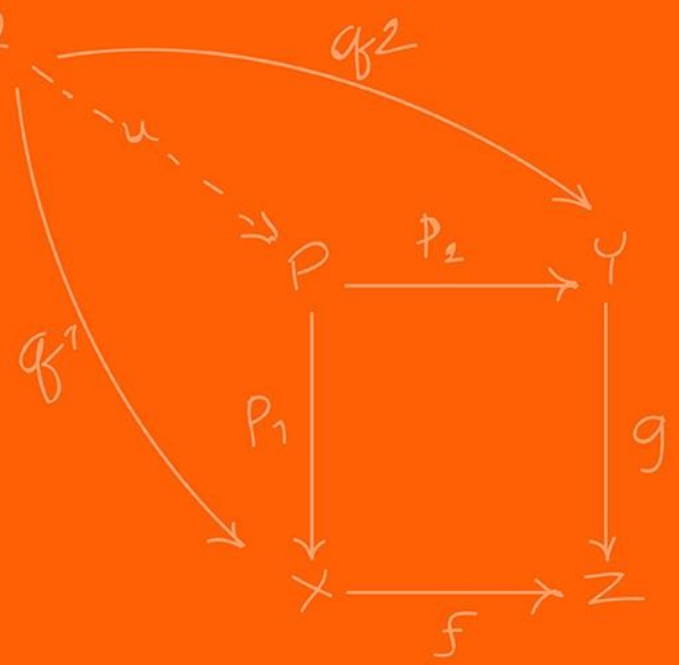
Path Planning – Movement Cost

```
frontier = PriorityQueue()
frontier.put(start, 0)
came_from = dict()
cost_so_far = dict()
came_from[start] = None
cost_so_far[start] = 0

while not frontier.empty():
   current = frontier.get()

   if current == goal:
      break

   for next in graph.neighbors(current):
      new_cost = cost_so_far[current] + graph.cost(current, next)
      if next not in cost_so_far or new_cost < cost_so_far[next]:
         cost_so_far[next] = new_cost
         priority = new_cost
         frontier.put(next, priority)
         came_from[next] = current
```





Heuristic search



Path Planning – Movement Cost

With Breadth First Search and Dijkstra's Algorithm, the frontier expands in **all** directions.

Why might that be a problem?

This is a reasonable choice if you're trying to find a path to **all** locations or to **many** locations.

However, a common case is to **find a path to only one location**.

Let's make the frontier expand **towards the goal** more than it expands in other directions.

Path Planning – Movement Cost

We need to **guide** the path planner's frontier expansion towards how would what is an intuitive way of doing this?

You try to minimize the distance to the goal, assuming there is no obstacle!

In Dijkstra's Algorithm we used the actual **distance from the start** for the priority queue ordering.

Here instead, in Greedy Best First Search, we'll use the **estimated distance to the goal** for the priority queue ordering.

The location closest to the goal will be explored first.

Path Planning – Movement Cost

```
frontier = PriorityQueue()
frontier.put(start, 0)
came_from = dict()
came_from[start] = None

while not frontier.empty():
   current = frontier.get()

   if current == goal:
      break

   for next in graph.neighbors(current):
      if next not in came_from:
         priority = heuristic(goal, next)
         frontier.put(next, priority)
         came_from[next] = current
```

Path Planning – Movement Cost

Those paths aren’t the shortest; however, this algorithm runs much faster when there aren’t a lot of obstacles.

The problem is that the paths aren’t as good, they are not **optimally the shortest** path like Dijkstra's and BFS.

Can we fix this?

Yes!

Here comes the magical A* algorithm!

Path Planning – Movement Cost

Dijkstra's Algorithm works well to find the **shortest path**, but it wastes time **exploring in directions that aren't promising**.

Greedy Best First Search explores in **promising directions,** but it may **not find the shortest path**.

A* is the combination of the two algorithm to get the best of both worlds, with no real downsides!

The A* algorithm uses both the **actual distance** from the start and the **estimated distance** to the goal.

It is extremely similar to Dijkstra's.

Path Planning – Movement Cost

```
frontier = PriorityQueue()
frontier.put(start, 0)
came_from = dict()
cost_so_far = dict()
came_from[start] = None
cost_so_far[start] = 0

while not frontier.empty():
   current = frontier.get()

   if current == goal:
      break

   for next in graph.neighbors(current):
      new_cost = cost_so_far[current] + graph.cost(current, next)
      if next not in cost_so_far or new_cost < cost_so_far[next]:
         cost_so_far[next] = new_cost
         priority = new_cost + heuristic(goal, next)
         frontier.put(next, priority)
         came_from[next] = current
```

This is the only new thing that was added!

Path Planning – Movement Cost

A* is the best of both worlds.

As long as the heuristic does not overestimate distances, A* finds an optimal path, like Dijkstra's Algorithm does.

A* uses the heuristic to reorder the nodes so that it's more likely that the goal node will be encountered sooner.

And … that's it! That's the A* algorithm.

Path Planning – Movement Cost

A* selects the next node to expand based on the function:

$$f(n) = g(n) + h(n)$$

- $g(n)$: The actual cost from the start node to node n.
- $h(n)$: The estimated cost from n to the goal (the heuristic).
- $f(n)$: The estimated total cost of the path through n to the goal.

Path Planning – Movement Cost

- Assume there is an optimal goal node G with a true minimum cost C^*.

Now, suppose A* is about to **terminate** by selecting a suboptimal goal node G_2 from the priority queue, where the cost $g(G_2) > C^*$.

For A* to be optimal, we must prove that the algorithm will **always find** and expand a node on the optimal path **before it ever reaches** G_2.

Path Planning – Movement Cost

Frontier assumption:

At any point before reaching the optimal goal, there must be at least one node n on the optimal path that is currently in the "Open Set" (the frontier).

Otherwise, it is impossible to reach the optimal goal since this would imply that no path node from the optimal goal is part of the frontier.

Any idea when this assumption might not happen?

This could happen if you have disconnected graphs and your start and end nodes are in different connected components.

Path Planning – Movement Cost

Heuristic underestimation assumption:

If the heuristic $h(n)$ is **admissible**, it satisfies:

$$h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost from n to the goal.

Path Planning – Movement Cost

- Because n is on the optimal path, we know that:

$$f(n) = g(n) + h(n)$$

Since $h(n) \leq h^*(n)$, then:

$$f(n) \leq g(n) + h^*(n)$$

By definition, $g(n) + h^*(n) = C^*$ (the optimal cost).

Therefore:

$$f(n) \leq C^*$$

Path Planning – Movement Cost

Now, consider our suboptimal goal G_2.

Since G_2 is a goal node, its heuristic $h(G_2) = 0$.

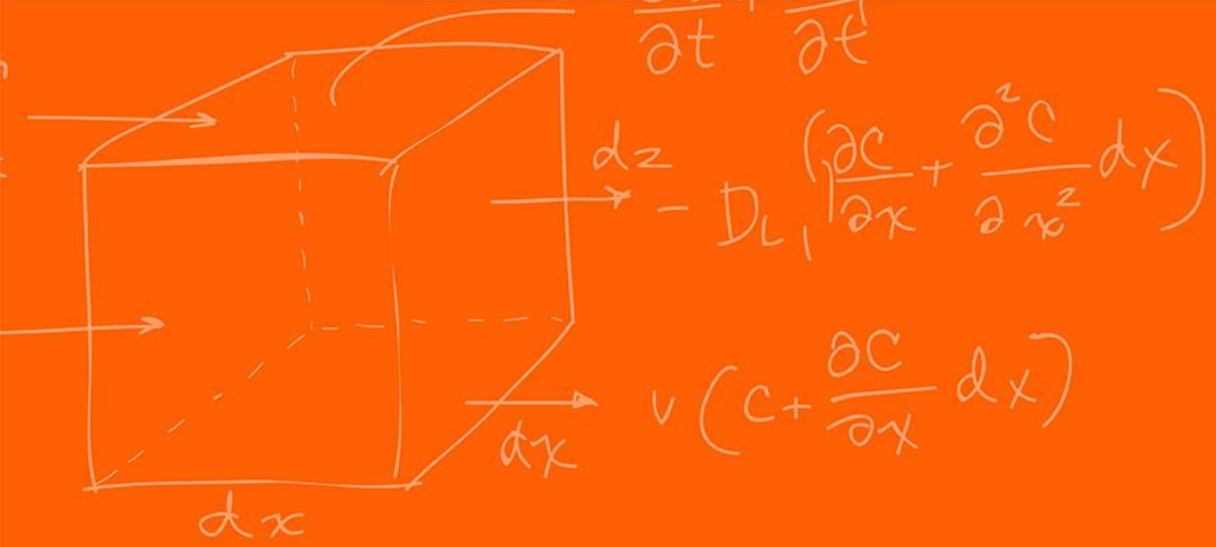
Thus:

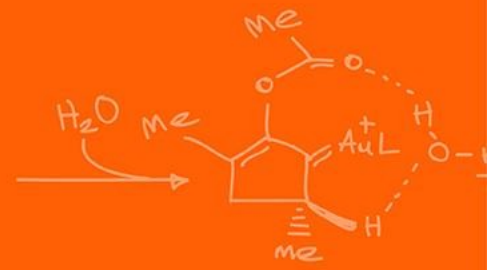
$$f(G_2) = g(G_2) + 0 > C^*$$

Because $f(n) \leq C^*$ and $f(G_2) > C^*$, it follows that: $f(n) < f(G_2)$

Since A* always expands the node with the **lowest f-value first**, it will always choose the node n on the optimal path before it ever picks the suboptimal G_2. This ensures the optimal path is found first.

Questions?





The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN